Is Hardware Offload Enough? Energy, Timing, and Scheduling of a Video Analytics Pipeline on the Raspberry Pi 4

Yazeed Alzughaibi
Department of Electrical and Computer Engineering
King Abdulaziz University, Jeddah, Saudi Arabia
yazeedmalzughaibi@gmail.com

***Abstract*—Single-board computers ship with hardware video codecs, and the usual advice for a video pipeline with deadlines is to use them. We test that advice on a Raspberry Pi 4 running a 720p decode–detect–overlay–encode pipeline with a person detector on every tenth frame. Moving decoding and encoding to the hardware codec raises sustained throughput from 18 to 30 frames/s. At equal work (15 frames/s) it cuts CPU time per frame by 52 %, but outlet energy per frame, measured in one session, by only about 16 %. Lower CPU time did not mean lower latency: in the way this application collects encoder output, each hardware-encoded packet was retrieved one or two frames after submission, so at 15 frames/s the median time from a frame's release to its encoded packet was 148 ms against 72 ms in software. With detection moved to a separate worker on one core at 15 frames/s, every hardware-mode frame-task and detection job met its CPU deadline under rate-monotonic priorities, with and without competing load, below an idealized response-time estimate; under the default Linux policy, competing load made every frame-task job late. Smaller findings concern slow CPU access to decoder buffers and the frequency governor.**

***Index Terms*—embedded systems, hardware offload, video processing, energy, DVFS, rate-monotonic scheduling, Raspberry Pi**

I. Introduction

A small board that watches a camera, finds people, and streams the annotated video is a common embedded workload. On a Raspberry Pi 4 the decoding and encoding can run on the CPU or on a dedicated video block, and the intuition is that the hardware codec should free the CPU, save energy, and make deadlines easier to meet.

We test that intuition by measurement: one pipeline with a software and a hardware codec path, a deadline for every frame, and repeated, balanced runs. The headline result agrees with the intuition; several details do not, and some would be easy to miss.

Our contributions are:

- A same-board comparison of software and hardware codec paths that separates throughput, CPU-task deadlines, and encoded-output latency, with CPU time and outlet power at equal work (Section IV).
- A measurement of what reading the hardware decoder's output costs the CPU (Section V).
- A frame-level comparison of the `ondemand` and `performance` governors (Section VI).
- A single-core version comparing an idealized fixed-priority analysis with measurement, and rate-monotonic with default Linux scheduling under competing load, with every detection job accounted for (Section VII).

The scripts, raw logs, analysis code, and a manifest mapping every reported number to its runs and the command that computes it are available at https://github.com/YazeedZee/ee654-pi-energy.

II. Background and Related Work

Hardware codecs. The Raspberry Pi 4's video block decodes and encodes H.264. Linux exposes it as a memory-to-memory V4L2 device, which FFmpeg wraps as `h264_v4l2m2m` [1]. Frames come back in buffers shared with the hardware.

Frequency scaling. Real-time dynamic voltage and frequency scaling (DVFS) work often assumes execution time scales inversely with frequency [2]; memory-bound work and high static power weaken that trade-off, so racing to idle can be as efficient as slowing down [3], [4].

Fixed-priority scheduling. For periodic tasks on one processor, rate-monotonic (RM) priority assignment gives shorter periods higher priority. The Liu and Layland bound $U \leq n(2^{1/n} - 1)$ (0.828 for two tasks) is a sufficient test [5]; when every period divides the longer ones (harmonic periods), RM schedules any such task set with $U \leq 1$. Response-time analysis (RTA) gives the worst-case response of each task from bounds on the execution times [6], [7]. The simplified models used here assume independent, preemptible tasks that do not suspend themselves. Linux implements fixed priorities as `SCHED_FIFO`; its default policy, `SCHED_OTHER`, is a fair-share scheduler, implemented since kernel 6.6 by Earliest Eligible Virtual Deadline First (EEVDF) [8], [9], to which the analysis does not apply. Tasks that wait for an accelerator are *self-suspending*, which complicates analysis [10].

Related work. Hardware and software decoding energy has been compared before, e.g., for HEVC [11], and a low-cost transcoding cluster has been built around the Raspberry Pi's hardware codec [12]. Slow CPU access to Raspberry Pi direct-memory-access (DMA) buffers has been reported, e.g., for camera buffers on a Pi 4 [13]. Marasini and Zhou cap the clock of a passively cooled Raspberry Pi 5 running a detector and report that a fixed maximum clock throttles heavily without a

fan [14]. We found no study measuring codec offload, buffer access, governor, and scheduling policy together; ours is a reproducible case study of their interaction on one board, not a new mechanism.

III. Platform and Method

Platform. All runs use one Raspberry Pi 4 Model B (rev. 1.4, 4 GB, four Cortex-A72 cores, 600–1800 MHz in 100 MHz steps) with Raspberry Pi OS (Debian 13), kernel 6.18.60 (`PREEMPT`, not `PREEMPT_RT`), FFmpeg 7.1.5, PyAV 14.2, and OpenCV 4.10. The board boots to a text console with non-essential services disabled. It has heatsinks and a 5 V fan, no case, and a USB-C charger. The firmware's under-voltage and throttle flags were clear in every sample of the runs behind Sections IV, VI and VII. The earlier diagnostic runs of Section V were made on a weaker USB supply; Section V states which of them were affected. Frequency is left to the default `ondemand` governor unless stated.

Pipeline v1. Fig. 1 shows the pipeline. A frame is decoded from H.264, converted from YUV to BGR, annotated with the latest boxes, converted back to YUV, and encoded at 4 Mbit s^{-1}. Every $N = 10$ frames a person detector (MobileNet-SSD [15], [16] on a 300×300 input through OpenCV's DNN module) runs first; in pipeline v1 it runs inline, so that frame waits for it. At 30 and 15 frames/s this is 3 and 1.5 detections per second. In *software mode* decoding and encoding run on the CPU (FFmpeg's H.264 decoder and `libx264`, preset `veryfast`, tune `zerolatency`); in *hardware mode* both use `h264_v4l2m2m`. Everything else is identical. Frame i is *released* at $t_0 + iT$, the scheduled start of its processing iteration, as if a camera delivered it then, and its deadline is one period later. The iteration starts at release or when the previous one ends, whichever is later, and asks the decoder for its next frame; the decoder is fed compressed packets from memory whenever it has none ready, so packets enter it ahead of their release, and time they spend inside the decoder before that is not measured. Late frames are not dropped, so a pipeline that cannot keep up builds a growing backlog. Pipeline v2, used only in Section VII, moves the detector into a separate worker thread.

Input. The input is the `pedestrian_area` sequence from the Xiph test-media collection [17] (people walking, static camera), re-timed to 30 frames/s, scaled to 720p, and encoded with `libx264` (High profile, CRF 20, one-second group of pictures (GOP)). The 375-frame clip is loaded into memory before each run and looped, so storage does not affect timing.

Timing metrics. Each stage is timed per frame with a monotonic clock. A frame's *frame-task response* runs from its release to the end of the CPU's work on it, when the frame has been submitted to the encoder; the frame *misses* if that exceeds one period. This is the job completion that scheduling analysis describes, not end-to-end latency: codecs decouple input from output [1], so the call that submits frame i may return packets of earlier frames. Only the bytes each encoder call returned were logged, so we *reconstruct* which call returned each frame's packet: assuming one packet per frame in submission order (neither encoder uses B-frames), the k-th call that returned output carried frame k. For the hardware encoder, the alignment of every key frame after the first with a large returned packet, in every run, was consistent with this mapping, but packet identities were not logged; the software encoder never returned an empty call, so its lag is zero throughout. We report the *encoder output lag* in frames and the *reconstructed packet-retrieval latency*, from release to the end of the call that returned the packet: an upper bound on when the packet was ready, describing this application's retrieval rather than the encoder's processing time. Neither includes decoder buffering. A frame's *service time* runs from the start of its first stage to the end of its last; for pipeline v1 we report a *load factor*, mean service time over the period. CPU time comes from per-process (v1) or per-thread (v2) accounting and can exceed service time, because the codecs and OpenCV run several threads. For pipeline v2 we use the utilization $U = \sum C_i/T_i$. A sampler reads the ARM clock, temperature, and throttle flags every 0.5 s; the pipeline logs the cpufreq frequency per frame.

Power. Power is read from a low-cost plug-in AC energy meter (MECHEER) between the outlet and the charger, so it includes the charger's losses and the fan. The meter displays 0.1 W steps and updates about once per second; its manual states no accuracy. Readings were taken by hand: the typical value per run for the equal-work comparison, and every value shown at about 5 s intervals for the governor comparison. Readings within a run are not independent (the display aliases with the detector's bursts), so the run is our unit of replication, and we compare only conditions measured in the same session.

Protocol. Conditions run in balanced order (A–B–B–A) with 30–60 s idle gaps. Runs last 90–180 s, the first 2–3 s are excluded, and every condition was run at least twice. This protocol applies to the principal comparisons (Sections IV, VI, and VII); the diagnostic runs of Section V (20 s pipeline runs and a 150-frame microbenchmark) are exceptions described there. At equal work, throughput, CPU time per frame, and load factor agreed within 1.5 % across eight runs per mode in four sessions; miss ratios and power varied more and are given as ranges. Platform state is recorded with every run.

IV. Offloading at the System Level

Throughput and deadlines at 30 frames/s. The software pipeline cannot keep up: it processes 17.8–17.9 frames/s at a load factor of 1.66, and every frame misses its deadline. The hardware pipeline sustains 30.0 frames/s at a load factor of 0.86; decoding falls from 13.4 to 0.3 ms per frame and encoding from 22.9 to 1.6 ms. Throughput is not deadline compliance, though. Each detection call takes about 110 ms inline, so the frame task misses its deadline for 71 % of hardware-mode frames, and the hardware encoder's packets are retrieved one frame after submission early in each run and two frames after it from then on (software: in the same call), so no frame's packet is retrieved within one period (median 125–129 ms). This is a throughput result, not a real-time one.

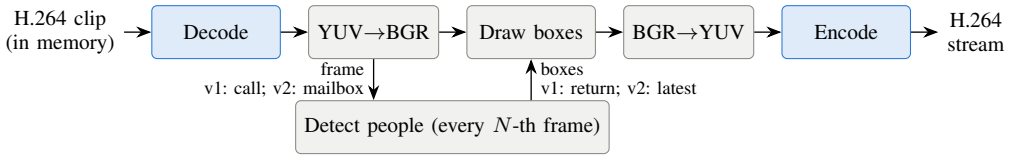


Fig. 1. The pipeline. Blue stages run on the hardware codec in hardware mode and on the CPU in software mode; gray stages always run on the CPU. In pipeline v1 (Sections IV–VI) the detector is called inline; in pipeline v2 (Section VII) it is a separate worker fed every N-th processed frame through a one-slot mailbox.

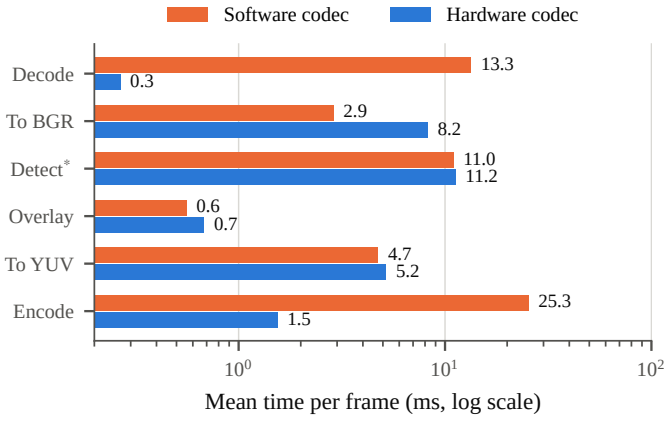


Fig. 2. Mean time per frame of each stage at 15 frames/s, pipeline v1 (two runs per mode, one session). *Detection runs on one frame in ten and is shown amortised per frame; one call takes 110–112 ms in either mode.

TABLE I
EQUAL WORK: PIPELINE V1 AT 15 FRAMES/S, $N = 10$

	Software	Hardware	Change
Four sessions, two runs per mode each			
CPU cores in use	2.17–2.18	1.04–1.06	−52%
CPU time per frame (ms)	145	70	−52%
Load factor	0.87	0.41	
Frame-task deadline misses	52–54%	20–21%	
Encoder output lag (frames)	0	1, then 2	
Packet retrieval, median (ms)†	70–75	148	
Encoded bitrate (Mb/s)	4.05–4.07	3.93	−3 to −3.5%
One session (two runs per mode), outlet power by hand			
Wall power (W; idle 3.6)	≈6.3	≈5.3	−1.0 W
Energy/frame, total (J)	≈0.42	≈0.35	≈ −16%
Energy/frame, above idle (J)	≈0.18	≈0.11	≈ −37%

†Release to reconstructed retrieval of the frame's encoded packet; frame-task misses end at encoder submission.

Equal work. At 30 frames/s the hardware pipeline does 70% more work, so we compare energy at 15 frames/s, which both modes sustain (Fig. 2, Table I). Hardware mode uses 70 ms of CPU per frame against 145 ms, a 52 % reduction, at a bitrate 3–3.5 % lower. The frame task still misses for 20–21 % of hardware-mode and 52–54 % of software-mode frames, because detection runs inline. Measured to the reconstructed retrieval of each frame's packet, the order reverses: the median is 148 ms in hardware mode, where the lag grew from one frame to two once per run, always at the call after an encoder key frame (in 6 of 8 runs the first key frame after the 375-frame clip first looped, so the input loop may play a part), against 72 ms in software mode. In this pipeline, offloading saved CPU time, not latency. The hardware run also ran cooler: in one matched pair of runs that started at 34.1 and 34.6 °C, it ended at 36.9 °C against 43.5 °C after 180 s.

Energy. The wall meter tells a more modest story. In the one session where we read it, the board idled at 3.6 W, the software pipeline drew about 6.3 W, and the hardware pipeline about 5.3 W, with both software runs above both hardware runs. Power above idle fell by about 37 % and total energy per frame by about 16 %, because more than half of the software pipeline's power is the idle level both modes pay. These are estimates from an uncalibrated meter, including the charger and fan, so we trust the direction more than the size. Such a gap between system and component savings is not new [11].

Cooling. Without the fan (case on), four cores at full load make the firmware throttle the clock, at times to 600 MHz. The pipeline is lighter: in a separate fanless session its 30 frames/s runs ended at 68–72 °C against 44–45 °C with the fan, the load factor changed by 1–4 %, and no run throttled.

V. The Frame-Access Trap

Our first hardware pipeline was *slower* than the software one. It converted each decoded frame to BGR directly from the decoder's buffer, and in the one pair of runs of that session without under-voltage throttling (detection every third frame, 20 s each) it managed 5.3 frames/s against 11.8 for software. The decode and encode stages were fast; the conversion took 143 ms per frame against 4 ms in software mode.

To find the cause, we timed three ways of getting a decoded 720p frame into BGR form, over 150 frames: a bulk copy of the frame's planes into ordinary memory; a conversion of that copy with OpenCV; and the direct conversion the pipeline used, libswscale through PyAV, reading straight from the decoder's buffer (Fig. 3). Bulk-copying a hardware-decoded frame is 2.5 times slower than copying a software-decoded one (595 against 1 483 MB s^{-1}), so the decoder's buffers are slower for the CPU to read, but only moderately. The direct conversion, however, takes 138 ms from a hardware-decoded frame against 3.9 ms from a software-decoded one: the same routine on the same frame size runs 35 times slower. Copying first and then converting takes 4.4 ms.

This diagnostic has two weaknesses. First, the copy-then-convert path used a different routine (OpenCV) from the direct path (libswscale), so the ratio $137.97/4.36 \approx 31.6$ mixes the copy with the change of routine, and the routines' similar

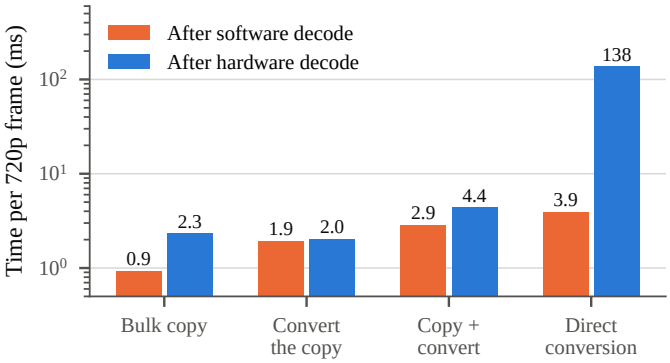


Fig. 3. Getting one decoded 720p frame into BGR form (median of 150 frames, diagnostic session). "Direct conversion" uses libswscale on the decoder's buffer; "convert the copy" uses OpenCV on a copy in ordinary memory.

speed on ordinary memory says nothing about the decoder's memory. Second, it ran on the weaker supply with no clock trace, after under-voltage and throttling had occurred that boot. Pipeline v2 (Section VII), which copies first and converts with libswscale, took 12.2 ms for this step in hardware mode (7.4 ms in software mode), but in another session, on another supply and on one core, so the copy's effect is not isolated; we did not run the paired comparison that would isolate it.

What the evidence does show is that the same routine was 35 times slower on the hardware decoder's buffer than on ordinary memory, while a bulk copy of that buffer was only 2.5 times slower. One unverified explanation is that the buffers are mapped without CPU caching, so a conversion's many small reads each pay a full memory access while a bulk copy reads in long bursts; slow CPU access to Raspberry Pi DMA buffers has been reported before [13]. All other results copy first; even so, the hardware path pays 8.2 ms for this stage in pipeline v1 against 2.9 ms in software.

VI. The Frequency Governor

At 15 frames/s the hardware pipeline leaves the CPU lightly loaded, and the `ondemand` governor responds by lowering the clock, briefly, to 700–800 MHz; the software pipeline never triggered a dip. We ran four trials in balanced A–B–B–A order: `ondemand`, `performance`, `performance`, `ondemand`. The `performance` governor requests the maximum frequency of 1.8 GHz. Table II summarizes the detection frames, the pipeline's longest.

The medians are nearly the same under both governors, and the tails differ less uniformly than a first look suggests. Under `performance`, no detection frame exceeded 144.1 ms. Under `ondemand`, run 1 had nine frames above 145 ms, up to 174.6 ms, and the frame-boundary frequency log shows all nine began at 900–1 100 MHz. Run 4 spent the most time at low clock but had only one such frame, for which the available frequency samples showed 1.8 GHz, and its detection frames that began or ended at low clock were not slower than the rest (median 124 against 125 ms). The evidence supports a governor effect on the tail in one of two runs, not a general rule, and frequency samples at frame boundaries and every 0.5 s are too coarse to attribute each slow frame with certainty.

TABLE II
Governor runs: hardware mode, 15 frames/s, in run order. Service times (ms) of the 177 detection frames per run

Run	Governor	Low clk	Median	p99	Max	# >145	Power (W)
1	`ondemand`	7%	121.6	164.3	174.6	9	5.50
2	`performance`	0%	121.5	138.6	142.8	0	5.12
3	`performance`	0%	121.4	137.9	144.1	0	5.38
4	`ondemand`	33%	125.2	143.9	147.2	1	5.09

Low clock: share of 0.5 s samples below 1.8 GHz. p99: 99th percentile. # >145: number of detection frames longer than 145 ms; the threshold was chosen after inspecting the data, just above the largest value under `performance`. Power: mean of 22–31 hand readings per run.

No energy difference was resolved either: the two runs of one governor differ by about 0.4 W, more than the governors differ, and two runs per governor allow no meaningful test. This is consistent with diminishing returns from frequency scaling on platforms with high static power [4]. For this workload, with a fan, neither `performance` run had a detection frame above 145 ms. That does not carry over to weaker cooling: the governor only requests the maximum clock, the firmware still lowers it when hot or under-powered, and on a passively cooled Raspberry Pi 5 a fixed maximum clock throttles heavily [14].

VII. Scheduling on One Core

Pipeline v1 runs detection inline on four cores, which makes classical analysis hard to apply. Pipeline v2, pinned to one core with one thread for OpenCV and for each codec, splits the work between a periodic *frame task* (decode to encoder submission, $T_f = 66.7$ ms at 15 frames/s) and a frame-triggered *detection worker* with nominal demand every ten frame periods ($T_d = 666.7$ ms). When the frame task processes a frame whose index is a multiple of ten, it places a copy in a one-slot mailbox, overwriting any job the worker has not yet taken; each frame draws the latest boxes. Deadlines equal periods, and a detection job's response is measured from its frame's release. Under RM (`SCHED_FIFO`) the frame task has priority 80 and the worker 70; the pipeline exits if it cannot set them. The alternative is the default `SCHED_OTHER`.

The execution time C of each job is its thread's CPU time. Each run computes the utilization and RTA from the largest C *observed in that same run*: a consistency check, not an independent prediction, and observed maxima are not proven worst-case execution times. We treat the analysis as an idealized reference model; it assumes independent, preemptible, periodic tasks without suspension, whereas here the worker is released by the frame task with some jitter, the threads share a mailbox and Python's interpreter lock, and part of the codec work runs outside them. Within the model the periods are harmonic, so the RM utilization limit is 1, not 0.828. Table III accounts for every nominal detection job.

CPU-task schedulability in the tested configuration. On one core, detection takes about 300 ms per call instead of

TABLE III
ONE CORE, 15 FRAMES/S, $N = 10$: TWO RUNS PER COLUMN (RANGES). DETECTION JOBS COUNTED AFTER A 3 S WARM-UP; 131 NOMINAL RELEASES PER RUN

	No competing load				Two background processes	
	HW, RM	HW, default	SW, RM	SW, default	HW, RM	HW, default
Frames/s	15.0	15.0	10.7	7.9	15.0	13.6–13.8
Frame-task deadline misses	0%	0%	100%	100%	0%	100%
Frame-task response p99	23 ms	41–42 ms	26 s	42 s	23 ms	7.1–8.4 s
C_f mean / max (ms)	19 / 24	19 / 24	93 / 116–118	95 / 119–120	19 / 24	18 / 22
C_d mean (ms)	301–303	302–306	293–295	297–298	302–303	309–311
C_d max (ms)	304–305	307–310	293–295	298–299	304–305	317–321
U with max C	0.82	0.82	2.2	2.2	0.82	0.80–0.81
RTA estimate, detection	498–501 ms	n/a	unsched.	n/a	497–498 ms	n/a
Detection jobs handed over	131	131	91–92	67	131	118–120
overwritten unstarted	0	0	90–91	0	0	48–49
completed (of which after run end)	131 (1)	131 (1)	1 (1)	67 (0–1)	131 (1)	70–71 (1–2)
completed on time	131	131	0	0	131	0
Detection response max	445–447 ms	445–448 ms	26–27 s	42.5–42.9 s	445–447 ms	8.9–10.0 s
Encoder output lag (frames)	2	1	0	0	2	1
Reconstructed packet retrieval, median	152 ms	100–101 ms	14 s	23 s	153 ms	3.8–4.6 s
Box age in video, median / max (frames)	11–12 / 16–17	11 / 16	none drawn	8 / 13	11 / 16	31 / 44
Box age at encoder submission, median	0.76–0.79 s	0.78 s	none drawn	23 s	0.76 s	5.8–6.5 s

"n/a": fixed-priority RTA does not apply to `SCHED_OTHER`. Frame-task responses and misses end at encoder submission. "After run end": completed after the timed run ended; when a run stops, the worker finishes its current job and any job still in the mailbox. Box age in video: frames between the analysed frame and the frame on which its boxes are drawn (one frame is 66.7 ms of video). Box age at encoder submission: from the analysed frame's release to the end of the frame task for the frame showing its boxes. Packet retrieval: reconstructed, see Section III.

110 ms on four. In hardware mode the frame task needed at most 24.4 ms, so in the reference model $U = 24.4/66.7 + 305.3/666.7 \approx 0.82$, below the harmonic limit of 1. RTA iterates $R = C_d + \lceil R/T_f \rceil C_f$ from $R = C_d + C_f$. With the unrounded maxima of run 1, $C_f = 24.421$ and $C_d = 305.345$ ms, it gives 329.8, 427.5, 476.3, and 500.7 ms, where it stops. The measured worst detection responses were 445–447 ms, below the estimate in every RM run, and no frame-task or detection job missed its deadline. The measured response includes 14–16 ms between a frame's release and the hand-over to the mailbox, a release jitter the model does not include; the estimate held despite it, but the model does not guarantee that it would.

These are deadlines for the CPU work. In every RM run each frame's packet was retrieved two frames after submission, constantly throughout the run, so the reconstructed retrieval latency was about 152 ms (at most 158 ms); under `SCHED_OTHER` without competing load the lag was one frame (about 100 ms). These figures describe when this application collected the packets, not how long the encoder needed. We did not determine why the lag depends on the policy; kernel work serving the codec may wait while the real-time threads run. An application with an end-to-end deadline would need this latency in its budget, or a design that collects encoder output as soon as it is ready.

In software mode the frame task alone needs 93 ms on average, more than its 66.7 ms period, so no policy can help, and the two fail in opposite ways. RM protects the frame task (10.7 frames/s, the most the core allows) and starves the detection worker: it never ran during the run, all but one handed-over job was overwritten, and that one completed after the run stopped. `SCHED_OTHER` lets the frame rate fall to 7.9 frames/s; it completed every job it was handed, but only 67 of 131 were handed over, none on time, and 42.4 s of the worst response of 43 s was frame backlog before hand-over. Because jobs are handed over only when frames are processed, an overloaded frame task also lowers the detection demand.

When the default policy is enough, and when it is not. We expected `SCHED_OTHER` to cause frame misses in the schedulable hardware case. It did not: the frame task sleeps most of each period, and the default scheduler serves a waking task promptly, so with nothing else to run it behaves much like RM. The difference shows in the margin (99th-percentile frame-task response 41–42 ms against 23 ms) and becomes decisive when other work shares the core. With two CPU-bound background processes, RM results were unchanged, because ordinary processes cannot preempt real-time ones. Under `SCHED_OTHER` the frame task received only a fair share of the core and fell to 13.6–13.8 frames/s, built a backlog until every frame was late (Fig. 4), 48–49 detection jobs were overwritten, and none of the 70–71 completed finished on time. The boxes on a frame came from a frame 31 frames earlier in the video (median; 11 under RM), released 5.8–6.5 s before that frame reached the encoder (0.76 s under RM). The reference model does not cover this case, since `SCHED_OTHER` provides neither fixed priorities nor isolation from other processes; it was an empirical baseline.

Suspension and blocking. A frame task waiting for the codec

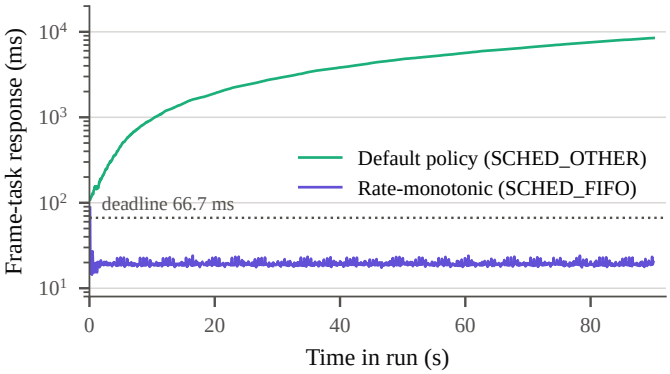


Fig. 4. Frame-task response times on one core with two background CPU-bound processes, hardware mode. The task set is identical; only the scheduling policy differs, and the response-time analysis applies only to the fixed-priority (RM) case.

would suspend itself [10], and both threads share Python's interpreter lock. Under RM, however, the largest gap between wall-clock service time and CPU time of any frame was 0.01 ms, and no frame started more than 0.09 ms late, over 5224 frames: no appreciable codec or lock blocking was observed in these runs, consistent with the decoder being fed ahead and packets being collected by later calls. These are observations, not bounds, and work by the video block or kernel threads on other cores is not in C.

VIII. Limitations

We used one board, clip, resolution, detector, software encoder preset (`veryfast`), and a Python pipeline, and did not compare output quality; our claim is that this configured software path misses its deadlines, not that every software path would. Packet retrieval was reconstructed from logged sizes, and decoder buffering was not measured. Power was read by hand from an uncalibrated outlet meter. The scheduling analysis is an idealized model checked against the runs that supplied its observed (not worst-case) execution times, on a kernel without `PREEMPT_RT`, without separating interrupt and driver work or recording real-time bandwidth settings. The frame-access diagnostic did not isolate the copy's effect. The governor comparison covers one load level with coarse frequency sampling, and the temperature difference one pair of runs.

IX. Conclusion

Moving the codec to hardware did what it is expected to do for throughput and CPU time: 30 instead of 18 frames/s, and half the CPU time at equal work. Everything else needed qualification. Hardware mode's CPU work met its deadlines only after detection became a separate, lower-priority worker, and even then, under RM, this application retrieved each frame's encoded packet two frames later. The outlet-energy saving was about a third of the CPU saving. The gain depended on how the CPU read the decoder's output. The default governor coincided with slow frames in one of two runs, for no resolvable energy benefit. And the idealized response-time analysis agreed with measurement under fixed priorities, while the default policy, which it does not cover, failed once other work shared the core. Before trusting an offload, check how the CPU reads the accelerator's buffers, when its output for a frame becomes available, how the governor treats bursty work under the available cooling, and whether the scheduling policy provides what the timing analysis assumes.

Acknowledgment

Generative AI was used in this work. Claude (Anthropic) helped write the experiment, pipeline, and analysis scripts, analyse the logs, and draft and revise the text of all sections. ChatGPT (OpenAI) reviewed drafts and suggested related work, which was then checked against the sources. The author designed and ran all experiments, reviewed all content, and takes responsibility for it.

References

[1] FFmpeg project, "FFmpeg," version 7.1. [Online]. Available: https://ffmpeg.org

[2] P. Pillai and K. G. Shin, "Real-time dynamic voltage scaling for low-power embedded operating systems," in *Proc. 18th ACM Symp. Operating Systems Principles (SOSP)*, 2001, pp. 89–102.

[3] A. Miyoshi, C. Lefurgy, E. Van Hensbergen, R. Rajamony, and R. Rajkumar, "Critical power slope: Understanding the runtime effects of frequency scaling," in *Proc. 16th Int. Conf. Supercomputing (ICS)*, 2002, pp. 35–44.

[4] E. Le Sueur and G. Heiser, "Dynamic voltage and frequency scaling: The laws of diminishing returns," in *Proc. Workshop on Power Aware Computing and Systems (HotPower)*, 2010.

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[6] M. Joseph and P. Pandya, "Finding response times in a real-time system," *The Computer Journal*, vol. 29, no. 5, pp. 390–395, 1986.

[7] N. Audsley, A. Burns, M. Richardson, K. Tindell, and A. J. Wellings, "Applying new scheduling theory to static priority pre-emptive scheduling," *Software Engineering Journal*, vol. 8, no. 5, pp. 284–292, 1993.

[8] I. Stoica and H. Abdel-Wahab, "Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation," Old Dominion Univ., Tech. Rep. TR-95-22, 1995.

[9] The Linux kernel documentation, "EEVDF scheduler." [Online]. Available: https://docs.kernel.org/scheduler/sched-eevdf.html

[10] J.-J. Chen *et al.*, "Many suspensions, many problems: A review of self-suspending tasks in real-time systems," *Real-Time Systems*, vol. 55, no. 1, pp. 144–207, 2019.

[11] M. Bey Ahmed Khernache, Y. Benmoussa, J. Boukhobza, and D. Menard, "HEVC hardware vs software decoding: An objective energy consumption analysis and comparison," *J. Systems Architecture*, vol. 115, art. no. 102004, 2021, doi: 10.1016/j.sysarc.2021.102004.

[12] P. Liu, J. Yoon, H. R. Kim, and S. Banerjee, "VideoCoreCluster: Energy-efficient, low-cost, and hardware-assisted video transcoding system," *Wireless Commun. Mobile Comput.*, vol. 2018, art. no. 7470234, 2018, doi: 10.1155/2018/7470234.

[13] Raspberry Pi Ltd., `picamera2` GitHub issue #740 (discussion of slow CPU copies from camera buffers). [Online]. Available: https://github.com/raspberrypi/picamera2/issues/740

[14] A. Marasini and Z. Zhou, "Sustainable edge vision via empirically calibrated DVFS: Eliminating thermal throttling on passively cooled hardware," arXiv:2609.04705, 2026.

[15] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[16] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 21–37.

[17] Xiph.Org Foundation, "Xiph.org video test media (derf's collection)." [Online]. Available: https://media.xiph.org/video/derf/